

FINDINGS

Deep security research — SSH-key challenge-response web login (helix-auth)

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Authority:** §11.4.150 (deep multi-angle research per change) + §11.4.99 (latest-source cross-reference) + §11.4.8 (deep-web-research-before-implementation) + §11.4.6 (no-guessing — sparse/contradictory sources flagged inline). **Scope:** security review of the helix-auth/auth_gate design in docs/superpowers/specs/2026-07-01-auth-pivot-ssh-key.md. READ + WRITE-A-DOC only. No code changed, no git. **Subject under review:** GET /login mints a server-HMAC-signed random nonce; user signs it with ssh-keygen -Y sign -n helixcode-login; POST /login verifies with ssh-keygen -Y verify against an allowed_signers file derived from the account's ~/.ssh/authorized_keys; success issues a signed-HMAC session cookie; runs non-root behind a Caddy HTTPS forward_auth gate; fail-closed.

Actionable hardening checklist for helix-auth (verifier stream applies these)

Ordered by severity. Each maps to a numbered angle below.

- **[A1-1] Enforce the namespace on BOTH ends AND pin it in allowed_signers.** Sign with -n helixcode-login, verify with the identical -n helixcode-login, AND emit each allowed_signers line with a namespaces="helixcode-login" restriction so a signature minted for any other namespace (git/email/file/another service reusing the same keys) is structurally rejected. Prefer a domain-qualified, versioned namespace — e.g. helixcode-login-v1@vasic.digital — per the OpenSSH-recommended NAMESPACE@YOUR.DOMAIN convention. (Angle 1, 2)
- **[A1-2] Do NOT trust the user-supplied principal for identity matching.** The SSHSIG signature does not carry an

identity — `-I` is an *assertion* the verifier makes. Use the SERVER-configured `HELIX_AUTH_PRINCIPAL` for `-I` and for every `allowed_signers` principals field; ignore/validate the form's principal to equal the configured value. Never let request input drive pattern-list matching. (Angle 1, 3, 7 — this is the direct lesson of CVE-2026-35414 SplitSSHell.)

- **[A1-3] Use a literal principal with NO commas and NO wildcards (*,?,!) in both `-I` and the `allowed_signers` principals field.** The principals field is a PATTERNS pattern-list; a wildcard or comma there widens who is accepted. (Angle 1, 7)
- **[A2-1] Server-issued, single-use, short-TTL, HMAC-bound challenge.** The nonce MUST be server-minted (≥ 32 bytes CSPRNG), carry a server HMAC over
nonce | issued-at | TTL | binding, expire in a short window (≤ 2 -5 min), and be consumed exactly once (replay cache / one-shot store). `ssh-keygen -Y verify` gives ZERO freshness guarantee — freshness is entirely the app's job. (Angle 2)
- **[A2-2] Feed verify the EXACT challenge bytes.** Show the user `printf %s '<challenge>' | ssh-keygen -Y sign ...` (no trailing newline — the spec already uses `printf %s`, keep it; `echo` would append `\n`). At verify, pipe the byte-identical challenge to `ssh-keygen -Y verify stdin`. A byte mismatch $\rightarrow$ verify fails (fail-closed is correct); never “normalize” or trim to make a mismatch pass. (Angle 1, 2)
- **[A2-3] Reject a client-supplied challenge.** Only accept challenges the server HMAC validates as its own + unexpired + unconsumed. Add login-CSRF defense on `POST /login` (SameSite cookie on the challenge state, per-request CSRF token, and Origin/Referer check). (Angle 2)
- **[A3-1] Convert `authorized_keys` $\rightarrow$ `allowed_signers` defensively:** parse each line, take ONLY keytype+base64 (drop options and the trailing comment), prepend the server principal, append the namespaces= restriction. Do not copy the comment or options verbatim. (Angle 3)
- **[A3-2] Key-type allow-list.** Accept `ssh-ed25519`, `sk-ssh-ed25519@openssh.com`, `sk-ecdsa-sha2-*`, `ecdsa-sha2-*`, `rsa-sha2-*/ssh-rsa` (RSA ≥ 3072 -bit). REJECT `ssh-dss` (DSA) and any unknown/weak type. (Angle 3)
- **[A3-3] Reject cert-authority lines in `authorized_keys`** when building `allowed_signers` (unless a CA login is explicitly intended and separately reviewed) — a CA key converted to a raw allowed-signer, or kept as cert-authority, broadens trust to every certificate that CA ever signs. (Angle 3, 7)
- **[A4-1] Never shell out through `sh -c`.** Use `Go exec.CommandContext` with an explicit argument slice; pass principal/namespace as discrete args (a leading-dash value is still a data arg, but see A1-3). Signature + `allowed_signers` are FILES (`-s`, `-f`), so no signature bytes reach `argv`. (Angle 4)
- **[A4-2] Temp-file hygiene.** Write the signature and the freshly-generated `allowed_signers` into files created under a private, `0700`-mode, per-request/per-process directory with `0600` file perms (`os.MkdirTemp/os.CreateTemp`), defer cleanup on every

path, and never in a world-writable/shared location. The `allowed_signers` file must not be attacker-writable. (Angle 4)

- **[A4-3] Bound the verify.** `context.WithTimeout` (e.g. 3-5 s) on the `ssh-keygen exec`; cap the accepted signature size (e.g. ≤ 16 KiB) and reject oversized/malformed input before spawning, to blunt DoS via huge/malformed signatures. Rate-limit POST / login (already planned) and keep failures generic + fail-closed. (Angle 4)
 - **[A5-1] Session cookie:** `__Host-` prefix, Secure, HttpOnly, SameSite=Strict (or Lax if strict breaks the flow), Path=/, no Domain; keep the HMAC integrity + short TTL. Add: **regenerate the session identifier on successful login** (anti session-fixation), support real logout/invalidation (already have /logout), and rotate/expire on TTL. (Angle 5)
 - **[A6-1] Consider a pure-Go SSHSIG verifier** (github.com/42wim/sshsig on golang.org/x/crypto/ssh) to eliminate the `exec` + temp-file surface entirely — but only if it verifies namespace + is well-maintained; otherwise shelling to the system `ssh-keygen` (with A4-*) is the conservative, spec-faithful choice. Decide explicitly; don't do both. (Angle 6)
 - **[A7-1] Require a patched OpenSSH (≥ 10.3 / 10.3p1)** on the host, because CVE-2026-35414 (comma-in-principal) and the OpenSSH 10.3 command-execution/scp fixes (CVE-2026-35386 class) land there. Record the host `ssh-keygen/ssh -V` version as captured evidence. (Angle 7)
-

Angle 1 — `ssh-keygen -Y sign / -Y verify` correctness pitfalls

How it actually works (OpenSSH man page, current): `-Y sign` signs stdin (or files) with the key from `-f`, and **requires** a namespace via `-n` “used to prevent signature confusion across different domains of use.” `-Y verify` reads the message on **stdin**, and requires: `-n` (namespace), `-s` (signature file), `-I` (signer identity), `-f` (`allowed_signers` file). The `allowed_signers` line format is `principals options keytype base64-key`; “the identity presented via the `-I` option must match a principals pattern in order for the corresponding key to be considered acceptable for verification.” The `principals` field is a **pattern-list** (PATTERNS in `ssh_config(5)`). ([OpenBSD ssh-keygen\(1\)](#), accessed 2026-07-01)

Namespace binding is the load-bearing control. SSHSIG prepends a 6-byte SSHSIG magic + the namespace to the signed blob; a raw SSH *authentication* signature starts differently, so “the SSH client and `ssh-keygen` will never produce identical signatures” — this is the cross-protocol firewall. The namespace stops a signature minted for one purpose (git commit signing, email, another service) being replayed as a login. ([Andrew Ayer](#) —

"It's Now Possible To Sign Arbitrary Data With Your SSH Keys",
accessed 2026-07-01; IETF draft-josefsson-sshsig-format-00,
accessed 2026-07-01)

Identity (-I) semantics — the subtle part. The SSHSIG blob contains the **public key + namespace only — NOT an identity**. -I is the verifier *asserting* "treat the signer as this principal"; verification succeeds when the signature's key appears in `allowed_signers` under a `principals` pattern that matches -I (and namespace/validity options pass). Therefore **the security does not come from -I; it comes from the key being in the allowed list**. Letting the *client* choose -I (as the spec's `POST /login {principal, signature}` currently does) is unnecessary and risky — it hands request input to pattern-list matching. (Man page + agwa blog, both above.)

-Y check-novalidate is NOT authentication. It "checks that a signature ... has a valid structure. This does not validate if a signature comes from an authorized signer." Never treat a check-novalidate pass as a login. Always use -Y verify with -I + -f. (OpenBSD ssh-keygen(1), accessed 2026-07-01)

Certificate vs raw-key handling. If a signature is made by a certificate, `allowed_signers` must mark the signer's CA key cert-authority, and "the expected principal name must match both the principals pattern in the allowed signers file and the principals embedded in the certificate itself." For a raw-key login gate you want raw-key lines only (see Angle 3). (OpenBSD ssh-keygen(1), accessed 2026-07-01)

Can -Y verify be "tricked into verifying different data"? Not cryptographically — the message is hashed inside the SSHSIG envelope and the namespace is bound in. The realistic failure mode is *operational*: the server verifies a message that is **not** the exact challenge it issued (e.g. a stale/attacker-chosen challenge, or a byte-differing challenge), which is a freshness/replay problem the crypto cannot catch (Angle 2), not a signature-forgery problem.

DO - Require -n <namespace> on sign AND verify; use a unique, domain-qualified, versioned namespace; additionally restrict via `namespaces=` in `allowed_signers` (defense in depth). - Set -I from server config, matched against a **literal** server-controlled principal. - Always use -Y verify (never check-novalidate) for the auth decision.

DON'T - Don't omit -n, and don't let sign/verify namespaces differ. - Don't accept a client-chosen -I/principal for matching. - Don't put commas or wildcards (*,?,!) in the principals field.

Angle 2 — Replay & challenge binding

Hard fact (honest gap flagged per §11.4.6): the agwa write-up and the man page **do not** provide any replay/freshness protection — `-Y verify` confirms “this key signed these exact bytes under this namespace” and nothing about *when* or *how many times*. (“The `-Y verify` command confirms cryptographic validity but provides no timestamp or freshness assurances,” per the agwa analysis, accessed 2026-07-01.) **All replay defense is the application’s responsibility.**

Binding pattern (matches precedent — Gitea/GitLab SSH-key verification all use a short-lived server token the user signs): (Gitea [“GPG/SSH signing”/verify flow](#) and [Gitea forum: “token is out-of-date”](#), accessed 2026-07-01 — Gitea signs a time-limited token with `ssh-keygen -Y sign -n gitea`; a slow user must restart because the token expires.)

- **Server-minted only:** nonce = ≥ 32 bytes from a CSPRNG. Attach a server HMAC over nonce | issued_at | ttl | optional client-binding. On POST, recompute + constant-time-compare the HMAC; reject anything the server didn’t mint. (The spec already plans “HMAC-signed by the server, bind to a short TTL + client” — keep it; this is the correct shape.)
- **Single-use:** record consumed nonces (or a signed one-time state) so a captured signature can’t be replayed within its TTL. Without single-use, replay within the TTL window is possible even with a valid HMAC.
- **Short TTL:** ≤ 2 -5 minutes. Balance against the human latency of running the local `ssh-keygen` command.
- **Optional client binding:** bind the challenge to the browser session (e.g. a SameSite state cookie or a hash of a per-visit token) so a challenge minted in attacker context can’t be completed in victim context.

Login-CSRF / cross-site on the POST: an attacker who can make the victim’s browser POST a *valid attacker-signed* login would log the victim into the attacker’s identity. Defenses: (1) the challenge state cookie is SameSite=Strict/Lax; (2) a synchronizer CSRF token tied to the GET `/login` render; (3) verify Origin/Referer. ([OWASP Session Management Cheat Sheet](#), accessed 2026-07-01.)

DO mint server-side, HMAC-bind, expire fast, consume once, feed byte-identical bytes to verify, add login-CSRF defenses. **DON’T** accept a client-supplied challenge, reuse a nonce, rely on `-Y verify` for freshness, or normalize/trim the challenge to force a match.

Angle 3 — authorized_keys -> allowed_signers conversion

The two formats are deliberately similar (“patterned after the AUTHORIZED_KEYS FILE FORMAT”) but **semantically different**: authorized_keys options (command=, restrict, no-pty, from=, permitopen=, ...) govern **interactive SSH sessions** and have NO meaning for signature verification; allowed_signers options are cert-authority, namespaces=, valid-after=, valid-before=. ([OpenBSD ssh-keygen\(1\)](#) + [sshd\(8\)](#) AUTHORIZED_KEYS FILE FORMAT, accessed 2026-07-01.)

Safe conversion recipe: 1. **Parse, don’t copy.** For each non-empty, non-# line: split off optional leading options, then read keytype base64 [comment]. **Emit only** <server-principal> namespaces="<ns>" <keytype> <base64>. Drop the comment and the SSH-session options entirely (they don’t translate — copying a comment/option verbatim risks smuggling a namespaces=/principal token or whitespace into the line you build). This closes the “comment/whitespace injection when building the allowed_signers line” vector. 2. **Multiple keys:** one output line per accepted key; all under the same literal server principal. Any of the account’s keys may then log in (matches “6 keys” in the spec). 3. **Key-type policy:** allow ssh-ed25519, sk-ssh-ed25519@openssh.com, sk-ecdsa-sha2-nistp256@openssh.com, ecdsa-sha2-*, rsa-sha2-256/512 / ssh-rsa with RSA modulus >= 3072-bit. **Reject ssh-dss/DSA** and anything unknown. (Modern OpenSSH already refuses DSA, but reject explicitly so behavior doesn’t depend on the host build.) 4. **Reject cert-authority lines** during conversion unless a CA-based login is an explicit, separately-reviewed feature — otherwise you silently broaden trust to every cert that CA signs (and this is the exact class the SplitSSHell CVE lives in — Angle 7). 5. **Principal hygiene:** the server principal you write is a fixed literal — **no commas, no PATTERNS wildcards**. This is what makes A1-2/A1-3 safe. 6. **Trust boundary honesty (§11.4.6):** ~/.ssh/authorized_keys is the account owner’s own file, so the owner is already trusted; the defensive parsing above is about not letting a malformed/hostile *line* corrupt the generated allowed_signers, and about not accidentally widening trust (DSA/CA/wildcards).

Contradiction/sparseness noted: community sources on allowed_signers focus on git-commit verification and repeatedly flag two *operational* gotchas — you can’t easily revoke a listed key, and a missing allowed_signers file makes verification hard-fail. ([dbushell](#), [Caleb Hearth](#), accessed 2026-07-01.) For helix-auth these translate to: (a) regenerate allowed_signers from authorized_keys on each start / on change so key removal takes effect (revocation-by-regeneration), and (b) fail-closed if the file is empty/absent (which the spec already mandates).

DO parse-and-rebuild, keytype allow-list, reject DSA/CA/wildcards, regenerate on change, fail-closed on empty. **DON'T** copy options/comments verbatim, honor SSH-session options as if they were signer options, or accept a CA/DSA key silently.

Angle 4 — Command-injection / temp-file / DoS safety when shelling to ssh-keygen

Go exec model. `exec.Command/exec.CommandContext` pass args as a slice directly to `execve` — there is **no shell**, so classic shell metacharacter injection does not apply as long as you never route through `sh -c "..."`. The documented rule: “avoid shell invocation altogether ... pass individual program arguments ... do not use `sh`, because internal protection does not work in this case.” ([Snyk — Go command injection](#); [Semgrep Go command-injection cheat sheet](#); [Sourcery](#), all accessed 2026-07-01.)

Residual argument-injection nuance. Even with a slice, a *value* that begins with `-` is a real gotcha for some CLIs. For `ssh-keygen -Y verify`, the attacker-influenced positions are `-I` `<principal>` and (via files) `-s/-f. -I <principal>` consumes the next argv as the value regardless of leading dash, so it can't “become another flag” — but combined with A1-2 you should not accept a client principal at all. The **signature never appears in argv** — it goes into the `-s` file — so a huge/garbage signature can't inject args; it can only cause parse work (see DoS). Where a CLI supports it, a `--` end-of-options terminator before positional/user values is a cheap extra guard.

Temp-file safety. - Create both the signature file and the generated `allowed_signers` under a per-process private dir: `os.MkdirTemp("", ...)` (0700), files 0600 (`os.CreateTemp`). Never a predictable path in `/tmp` shared world-writable — that invites a symlink/TOCTOU swap of the `allowed_signers` (which would let an attacker substitute the trusted key set). - defer remove the dir on **every** return path (success and error) — §11.4.14 cleanup discipline. - The `allowed_signers` file is trust-critical: generate it yourself with 0600, never read one an attacker could have written.

DoS / resource limits. - Wrap the `exec` in `context.WithTimeout` (~3-5 s) so a hung/slow `ssh-keygen` can't pin a worker; kill on timeout. - Reject the input before spawning if the pasted signature exceeds a sane cap (e.g. 16 KiB) or fails a cheap `-----BEGIN SSH SIGNATURE-----` shape check. - Keep the existing rate-limiter on `POST /login`; return a generic 401 on all failures (no oracle about *why* it failed).

DO slice-args (no sh -c), private 0700 temp dir + 0600 files, defer cleanup, CommandContext timeout, size-cap + rate-limit, generic errors. **DON'T** build a shell string, put user bytes in argv positions that could be read as flags, write temp files to shared/predictable paths, or run ssh-keygen without a timeout.

Angle 5 — Session cookie hardening

The spec already plans HttpOnly/Secure/SameSite/HMAC/TTL — good. Add, per [OWASP Session Management Cheat Sheet](#) (accessed 2026-07-01) and [OWASP WSTG cookie-attributes](#) (accessed 2026-07-01):

- **__Host- name prefix.** Set-Cookie: __Host-helixid=...; Path=/; Secure; HttpOnly; SameSite=Strict. The prefix forces Secure, Path=/, and **no Domain** — “integrity against network-based session fixation.” Since Caddy terminates HTTPS, Secure + __Host- are satisfiable.
- **SameSite=Strict** preferred (this is a single-origin editor gate, so Strict rarely breaks anything); fall back to Lax only if a cross-site entry link is required. Never None without Secure; never rely on the browser default.
- **Session-fixation defense (currently unstated in the spec):** regenerate / re-mint the session identifier on successful login, and invalidate any pre-auth session — “invalidate the old session and create a new one after login.” With a self-contained HMAC token this means: issue a *fresh* token at login bound to a fresh session-id, and don't accept any pre-login token as post-login.
- **Logout + rotation:** /logout must actually invalidate (short TTL + server-side revocation list if you need instant kill; otherwise rely on TTL). Rotate the cookie-signing secret on a schedule and on suspected compromise (§11.4.10).
- **HMAC token content:** keep it minimal + integrity-protected (e.g. subject | issued-at | expiry, HMAC-SHA256, constant-time verify). Don't store anything the client shouldn't see; don't trust any field before HMAC check.

DO __Host- + Secure + HttpOnly + SameSite=Strict + short TTL + regenerate-on-login + real logout. **DON'T** ship a Domain= cookie, skip session regeneration, accept pre-auth tokens post-auth, or use SameSite=None.

Angle 6 — Alternatives / precedent (shell-out vs Go-native)

Precedent — everyone uses the same challenge-sign-verify shape. Gitea and GitLab both verify a user's SSH key on the web by having the user sign a short-lived server token with `ssh-keygen -Y sign -n <service-namespace>` and pasting the `-----BEGIN SSH SIGNATURE----- blob`; the server verifies it. This validates the helix-auth approach as mainstream. ([Gitea signing docs](#); [Tech Addressed Gitea verify walkthrough](#); [GitLab SSH signed-commits docs](#), accessed 2026-07-01.) Broader identity systems (HashiCorp Vault SSH secrets engine, smallstep, Teleport) issue **short-lived SSH certificates** rather than verifying ad-hoc SSHSIG blobs — a heavier model than needed for a single-account editor gate, but the direction to grow toward if multi-user/expiry/revocation becomes a requirement.

Shell-out to ssh-keygen (spec's choice) - Pros: uses the exact, audited OpenSSH implementation (bug-for-bug identical to what the user runs); zero crypto to get wrong in-house; trivially matches `-Y sign` output; easy to test with the real key (spec's §11.4.98 autonomous path). - **Cons:** process-spawn + two temp files per login (the Angle-4 surface); depends on host OpenSSH version (Angle 7); harder to unit-test in isolation; per-call latency.

Go-native SSHSIG (github.com/42wim/sshsig) - Pure Go on golang.org/x/crypto/ssh; `Sign(privkey,data,namespace) / SignWithAgent(...)` and verification; output compatible with `ssh-keygen -Y sign`; derived from Sigstore Rekor's SSH PKI. ([42wim/sshsig](https://42wim/sshsig/pkg.go.dev/github.com/42wim/sshsig); pkg.go.dev/github.com/42wim/sshsig, accessed 2026-07-01.) - **Pros:** no exec, no temp files, no host-OpenSSH dependency, in-process timeouts, easier tests. - **Cons/caveats (§11.4.6):** golang.org/x/crypto/ssh itself "does not yet support SSH file signing," so you're trusting a third-party lib's SSHSIG parsing/namespace-checking; the README does not explicitly document an `allowed_signers`-style trusted-key API, namespaces=restriction handling, or a security-audit status. If you adopt it you **MUST** verify it enforces namespace binding and does key-set matching yourself, and pin+review the version (§11.4.74 catalogue-check).

Recommendation: default to shelling to the system `ssh-keygen` (conservative, spec-faithful, real-crypto) with the Angle-4 hardening; treat Go-native as an optimization only after auditing that `sshsig` enforces the namespace + key allow-list. Do NOT run both paths. (golang.org/x/crypto/ssh docs, accessed 2026-07-01.)

Angle 7 — Known CVEs / advisories relevant to SSH signature verification

CVE-2026-35414 — “SplitSSHell” (comma-in-principal access-control bypass). In OpenSSH 5.6 through 10.2p1, sshd’s `authorized_keys` `cert-authority,principals=` path used `match_list()` (comma-splitting, built for algorithm negotiation) where `strcmp()` was correct, so a certificate principal like `deploy,root` was split into `deploy + root`; if `root` was allowed, auth succeeded though no principal equals `root` exactly. CVSS ~8.1; **fixed in OpenSSH 10.3/10.3p1.** (CIS advisory 2026-040; [cve.news CVE-2026-35414](#); [Cyera SplitSSHell root-cause writeup](#), all accessed 2026-07-01.)

Direct relevance to helix-auth (honest scope note, §11.4.6): this CVE is in sshd’s `authorized_keys` **cert path**, NOT in `ssh-keygen -Y verify`. But `helix-auth` lives in the *same design space* — matching a signer identity against a principals **pattern-list**. The transferable lessons are exactly A1-2/A1-3/A3-5: **never let untrusted input drive pattern-list matching; treat identities as opaque atomic strings; keep the server-controlled principal a literal with no commas/wildcards; reject cert-authority conversion.** Following those, `helix-auth` avoids the whole comma/pattern-splitting hazard class regardless of host OpenSSH version — but you should still require `>= 10.3` on the host.

OpenSSH 10.3 (2026-04) hardening bundle. 10.3 also fixed a `scp/command-execution/privilege-escalation` class (reported around CVE-2026-35386 and siblings) and dropped legacy rekeying. Running `>= 10.3` gets all of these. ([Help Net Security — OpenSSH 10.3](#); [securityonline.info — OpenSSH 10.3 / CVE-2026-35386](#), accessed 2026-07-01.)

No SSHSIG-specific -Y verify CVE found. An enumerated search across CIS, [cve.news](#), OpenSSH release notes and the man page surfaced **no** CVE specific to `ssh-keygen -Y verify / allowed_signers` signature verification as of 2026-07-01. Per §11.4.6/§11.4.118 this is “none found in the enumerated sources,” not a proof of “none exists” — re-verify at each release boundary.

DO require OpenSSH `>= 10.3` on the host, record its version as evidence, and apply the atomic-identity/literal-principal lessons. **DON’T** enable CA-based (`cert-authority`) signer conversion, use comma/wildcard principals, or run an OpenSSH `< 10.3` build under this gate.

Sources verified 2026-07-01

- OpenBSD ssh-keygen(1) manual — <https://man.openbsd.org/ssh-keygen.1>
- man7 ssh-keygen(1) — <https://man7.org/linux/man-pages/man1/ssh-keygen.1.html>
- Andrew Ayer, “It’s Now Possible To Sign Arbitrary Data With Your SSH Keys” — https://www.agwa.name/blog/post/ssh_signatures
- IETF draft-josefsson-shsig-format-00 (SSH SIG format) — <https://www.ietf.org/archive/id/draft-josefsson-shsig-format-00.html>
- CVE-2026-35414 — CIS advisory 2026-040 — https://www.cisecurity.org/advisory/a-vulnerability-in-openssh-could-allow-for-authentication-bypass_2026-040
- CVE-2026-35414 — cve.news — <https://www.cve.news/cve-2026-35414/>
- Cyera “SplitSSHell” root-cause analysis — <https://www.cyera.com/research/splitsshell-when-a-comma-becomes-root-how-a-single-character-broke-openssh-certificate-authentication>
- Help Net Security — OpenSSH 10.3 released — <https://www.helpnetsecurity.com/2026/04/02/openssh-10-3-released/>
- securityonline.info — OpenSSH 10.3 / CVE-2026-35386 — <https://securityonline.info/openssh-10-3-security-patch-cve-2026-35386-vulnerabilities/>
- OWASP Session Management Cheat Sheet — https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP WSTG — Testing for Cookie Attributes — https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/06-Session_Management_Testing/02-Testing_for_Cookies_Attributes
- Snyk — Understanding Go command-injection vulnerabilities — <https://snyk.io/blog/understanding-go-command-injection-vulnerabilities/>
- Semgrep — Go command-injection cheat sheet — <https://semgrep.dev/docs/cheat-sheets/go-command-injection>
- Sourcery — Go command-line argument injection — <https://www.sourcery.ai/vulnerabilities/command-line-arguments-go>
- Gitea — GPG/SSH signing docs — <https://docs.gitea.com/administration/signing>
- Gitea forum — “token is out-of-date” (time-limited challenge) — <https://forum.gitea.com/t/the-provided-ssh-key-signature-or-token-do-not-match-or-token-is-out-of-date/7866>
- Tech Addressed — Add & verify SSH keys in Gitea — <https://www.techaddressed.com/tutorials/add-verify-ssh-keys-gitea/>
- GitLab — Sign commits with SSH keys — https://docs.gitlab.com/user/project/repository/signed_commits/ssh/

- 42wim/sshsig (Go SSHSIG library) — <https://github.com/42wim/sshsig>
- pkg.go.dev — [github.com/42wim/sshsig](https://pkg.go.dev/github.com/42wim/sshsig) — <https://pkg.go.dev/github.com/42wim/sshsig>
- pkg.go.dev — [golang.org/x/crypto/ssh](https://pkg.go.dev/golang.org/x/crypto/ssh) — <https://pkg.go.dev/golang.org/x/crypto/ssh>
- dbushell — Verify signed git commits (allowed_signers) — <https://dbushell.com/2023/06/20/git-ssh-verify-allowed-signers/>
- Caleb Hearth — Signing git commits with your SSH key — <https://calebhearth.com/sign-git-with-ssh>